31. What happens when features are highly correlated in Lasso?

When input features are highly correlated (a condition known as multicollinearity), Lasso regression (L_1 regularization) exhibits a specific, unstable behavior:

- **Arbitrary Selection:** If a group of features are highly correlated with each other and are all predictive of the target variable, Lasso's optimization path will typically pick **one** feature from that group completely at random and assign it a large weight, while driving the coefficients of all the other remaining correlated features to **exactly zero**.
- **High Variance in Model Selection:** Because this selection is highly sensitive to minor statistical fluctuations, small changes or noise in the training dataset can cause Lasso to drop the previously selected feature and pick a different correlated one instead.
- **Loss of Group Information:** Unlike Ridge regression—which retains all correlated features and shrinks their weights proportionally together—Lasso breaks the group structure, making model interpretation highly volatile when dealing with highly collinear data.

32. What is the advantage of combining L1 and L2 penalties in Elastic net?

Elastic Net combines the penalties of both Lasso (L_1) and Ridge (L_2) regression into a single unified cost function:

$$J(\boldsymbol{\beta}) = \sum_{i=1}^{m} \left(y_i - \boldsymbol{x}_i^T \boldsymbol{\beta}\right)^2 + \alpha\rho \sum_{j=1}^{n} |\beta_j| + \frac{\alpha(1-\rho)}{2} \sum_{j=1}^{n} \beta_j^2$$

Where α controls the total regularization strength, and ρ is the mixing hyperparameter (controlling the ratio between L_1 and L_2).

Advantages of Combining the Penalties

1. **Overcoming Lasso’s Correlated Feature Problem (The Grouping Effect):** By incorporating the L_2 (Ridge) penalty, Elastic Net encourages highly correlated features to be shrunken and selected **together** as a group, rather than arbitrarily picking one and discarding the rest like Lasso.
2. **Preserving Sparsity (Feature Selection):** Thanks to the L_1 component, Elastic Net retains the ability to drive completely irrelevant feature weights to exactly zero, giving you a sparse, simplified model.
3. **Robustness Over Extreme Dimensionality:** In scenarios where the number of features (n) is far greater than the number of training samples (m), Lasso is mathematically capped at

selecting at most m features. Elastic Net bypasses this limitation, allowing the model to select more than m features while maintaining structural stability.

33. What is K-Means clustering? How are initial centroids chosen in K-Means? Why do we iterate in K-Means? Why is K-Means sensitive to initial centroids? Why do we need feature scaling in K-Means? What type of cluster shapes does K-Means work well on?

Definition

K-Means is an unsupervised partition-based clustering algorithm that groups a dataset into K distinct, non-overlapping clusters based on feature proximity.

Centroid Initialization

- **Random Initialization:** Standard K-Means randomly selects K data points from the dataset to act as the starting coordinates for the cluster centers (centroids).
- **K-Means++ Initialization:** To optimize this process, K-Means++ picks the first centroid at random, then selects subsequent centroids from the remaining data points with a probability proportional to their squared distance from the closest existing centroid. This spaces out the initial centers across the feature area.

Why We Iterate

K-Means utilizes an iterative expectation-maximization approach because finding the absolute global minimum partition across all data points simultaneously is computationally intractable (NP-hard). The algorithm must iterate through alternating steps to converge toward a stable local minimum:

1. **Assignment Step:** Assign each data point to its closest centroid using Euclidean distance.
2. **Update Step:** Recalculate the position of each centroid by taking the arithmetic mean of all data points currently assigned to that cluster. This loop runs until the centroids stop shifting position or assignments stabilize.

Sensitivity to Initial Centroids

Because the optimization landscape is non-convex and full of local minima, the final layout of the clusters is entirely dependent on where the starting centroids are placed. If two initial centroids are placed within the same natural dense cluster, the algorithm can easily get trapped in a poor local configuration, splitting a single coherent group and mixing other distinct groups together.

Why Feature Scaling is Mandatory

K-Means relies heavily on mathematical distance metrics (like Euclidean distance) to evaluate similarity. If one feature has a massive numerical range (e.g., Income from 0 to $1,000,000) and another feature has a tiny range (e.g., Age from 0 to 100), the distance calculation will be completely dominated by the high-magnitude scale. Feature scaling balances all variables onto an equal footing, preventing individual features from skewing the geometric space.

Ideal Cluster Shapes

K-Means works optimally only on clusters that are **spherical (globular), compact, and roughly equal in size and density**. It fails completely on elongated, anisotropic, or concentric ring-like data shapes because its distance thresholds naturally slice up spaces into linear Voronoi cells.

34. What is the difference between DBSCAN and K-Means? What is ε (epsilon) in DBSCAN? Why is DBSCAN good for detecting outliers?

Core Differences

Feature Property	K-Means Clustering	DBSCAN Clustering
Algorithmic Approach	**Partition-based**: Divides the data into a pre-defined number of clusters based on distance to a center.	**Density-based**: Groups regions of high point-density together, separated by expansive low-density zones.
Number of Clusters (K)	Must be explicitly defined by the user beforehand.	Automatically discovers the number of clusters based on the data structure.
Cluster Shape Flexibility	Only handles spherical/convex clusters well.	Easily captures complex, highly arbitrary geometric shapes (e.g., lines, rings, crescents).
Outlier Handling	Highly sensitive to outliers; anomalies pull centroids away from their true positions.	**Immune to noise**; isolates and labels sparse anomalies explicitly as noise.

The Epsilon (ε) Parameter

In DBSCAN, ε **(epsilon)** defines the radius of the neighborhood around any given data point. It establishes the physical boundary within which the algorithm counts neighboring points to

evaluate local data density.

- If ε is too small, large parts of valid clusters will be mislabeled as noise.
- If ε is too large, completely distinct clusters will merge into a single group.

Why DBSCAN is Ideal for Outlier/Noise Detection

DBSCAN defines clusters based on structural density requirements: a point must have a minimum number of neighbors (`minSamples`) within its ε-radius to be considered part of a cluster core.

If a data point lies in a sparse, low-density region and does not have enough neighbors within its ε-neighborhood—and cannot be reached from the edge of any dense cluster core—DBSCAN leaves it unassigned, flagging it explicitly as **noise** (1 or an outlier). This makes it an effective tool for filtering anomalies out of a dataset.

35. What are the types of encoding that you know? When should we use one-hot encoding instead of label encoding? What problem can arise from label encoding?

Common Types of Categorical Encoding

- **Label Encoding / Ordinal Encoding:** Assigns a unique, sequential integer to each unique categorical text label (e.g., `Red` $\rightarrow 0$, `Green` $\rightarrow 1$, `Blue` $\rightarrow 2$).
- **One-Hot Encoding (OHE):** Converts a single categorical column into N distinct binary columns (where N is the number of unique categories). Each column contains a 1 for rows matching that category and a 0 for all others.
- **Target Encoding:** Replaces categorical values with the mean target value of that corresponding category calculated across the training set.

When to Use One-Hot Encoding Over Label Encoding

One-Hot Encoding should always be used when dealing with **nominal categorical variables**—categories that possess **no inherent mathematical ordering or ranking** (e.g., countries, car brands, or colors).

Label encoding should be reserved exclusively for **ordinal variables**, which have a clear structural hierarchy (e.g., Education levels: `High School` $\rightarrow 0$, `Bachelors` $\rightarrow 1$, `PhD` $\rightarrow 2$).

The Core Problem with Label Encoding Nominal Data

If you apply label encoding to nominal categories (such as colors), you introduce an artificial mathematical order that does not exist in reality (e.g., `Red (0) < Green (1) < Blue (2)`).

When distance-based or regression-based machine learning algorithms digest these arbitrary numerical values, they interpret them as continuous metrics. The model will incorrectly assume that `Blue (2)` is twice as far or twice as important as `Green (1)`, causing distorted calculations, poor linear fits, and compromised predictive performance.

36. What is feature scaling? Which algorithms are sensitive to feature scaling?

Definition

Feature scaling is a data preprocessing step that standardizes or normalizes the independent numerical variables of a dataset, bringing their mathematical ranges onto a uniform, identical scale without altering the underlying relationships in the data.

Algorithms Highly Sensitive to Feature Scaling

1. **Distance-Based Algorithms:** Algorithms like k**-Nearest Neighbors (KNN)** and **K-Means Clustering** calculate geometric distances (Euclidean/Manhattan) to evaluate similarity. Unscaled features with large absolute ranges will dominate the equations, rendering smaller-scale features irrelevant.
2. **Gradient Descent-Based Optimizers:** In models like **Logistic Regression** , Linear Regression, and **Neural Networks**, disparate feature scales warp the error landscape into an elongated, asymmetric valley. This causes gradient descent updates to oscillate wildly, delaying or preventing convergence.
3. **Support Vector Machines (SVM):** SVMs seek to maximize the physical margin separating classes. If features have unbalanced scaling, the support vector coordinates will skew the hyperplane orientations along the axes of the largest numerical ranges.

> **Algorithms NOT Sensitive to Scaling:** Tree-based ensemble models (such as Decision Trees and Random Forests) are completely invariant to feature scaling. They perform localized, vertical splits on one feature at a time, making their mathematical partitions independent of the scales of other features.

37. What is normalization (Min-Max scaling)? What is standardization (Z-score scaling)? What happens if features are not scaled?

Normalization (Min-Max Scaling)

Normalization shifts and rescales the data so that all feature values fall strictly within a bounded range, typically between 0 and 1. It is calculated as:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- **When to use:** Use normalization when you need bounded ranges (e.g., image pixel values from 0 to 255 scaled down for deep learning) or when you know your data does not follow a Gaussian distribution.

Standardization (Z-Score Scaling)

Standardization centers the data around a mean of 0 and rescales it to achieve a standard deviation of 1. It does not bind the features to a strict maximum or minimum limit. It is calculated as:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

Where μ is the feature mean and σ is its standard deviation.

- **When to use:** Use standardization when your machine learning algorithms assume normally distributed data, or when the data contains extreme outliers, as it does not compress outliers into a tiny, uninformative interval like Min-Max scaling.

Consequences of Leaving Features Unscaled

If features are left unscaled, optimization routines like gradient descent will slow down or fail to find an optimal solution due to uneven gradient fields. Distance-dependent classifiers will misclassify points by ignoring lower-magnitude features, and regularization methods (like Ridge or Lasso) will unfairly penalize features with smaller absolute numbers simply because of their measurement units.

38. What is multicollinearity? Why is multicollinearity a problem in regression? How can multicollinearity be detected? Why does multicollinearity not always affect prediction accuracy? How can correlated features be detected?

Definition

Multicollinearity is a statistical phenomenon in which two or more independent explanatory features in a multiple regression model are highly linearly correlated with one another, making them redundant.

Why it is a Severe Problem for Regression

1. **Unstable, High-Variance Coefficients:** Multicollinearity severely destabilizes the Ordinary Least Squares (OLS) calculation matrix $(\boldsymbol{X}^T\boldsymbol{X})^{-1}$. It inflates the variance of the estimated coefficients, meaning tiny shifts in the training data can cause model weights to swing erratically or change signs completely.
2. **Loss of Interpretability:** It makes it impossible to isolate the individual effect of a single feature on the target variable. You cannot accurately determine which variable is truly driving predictions because they move together, obscuring their independent statistical significance.

Detection Methods

- **Correlation Matrix / Heatmaps:** Visually inspecting a Pearson correlation matrix to spot feature pairs with coefficients near ± 1.
- **Variance Inflation Factor (VIF):** A metric that measures how much the variance of an estimated regression coefficient is increased due to collinearity. A VIF value above 5 or 10 indicates severe multicollinearity that requires remediation.

Impact on Predictive Accuracy

Multicollinearity does not necessarily degrade a model's **overall prediction accuracy or validation scores**. If the correlated feature relationships remain identical in both the training data and the unseen testing data, the model can still generate accurate predictions. The issue is structural and explanatory; it ruins parameter interpretability and weight stability, but the collective predictive signal can still yield an accurate output vector.

39. What is eigenvalue and eigenvector in PCA? What are the steps involved in PCA? How does PCA reduce multicollinearity? Can PCA handle nonlinear data? When would you prefer feature selection over PCA?

Eigenvalues and Eigenvectors in PCA

Principal Component Analysis (PCA) is a linear dimensionality reduction technique.

- **Eigenvector:** Represents the directional axes of the new feature subspace (the Principal Components) along which the data is projected. They define the orientation of the maximum variance paths.
- **Eigenvalue:** A scalar value that quantifies the amount of variance variance retained along its corresponding eigenvector direction. A larger eigenvalue means its principal component carries more information.

Step-by-Step PCA Protocol

- **Step 1: Standardization:** Standardize the original features to have a mean of 0 and a standard deviation of 1 to prevent features with larger scales from dominating the variance calculations.
- **Step 2: Covariance Matrix Construction:** Compute the $N \times N$ covariance matrix to capture the linear relationships and variances between all feature pairs.
- **Step 3: Eigendecomposition:** Calculate the eigenvectors and corresponding eigenvalues of the covariance matrix.
- **Step 4: Sorting Components:** Sort the eigenvectors in descending order based on their eigenvalues.
- **Step 5: Feature Projection:** Select the top K eigenvectors to form a transformation matrix, and multiply it by the original data matrix to project the dataset into a lower-dimensional subspace.

Reducing Multicollinearity

PCA targets the underlying cause of multicollinearity by design. Because eigenvectors are mathematically orthogonal (perpendicular) to each other, **the resulting principal components are completely uncorrelated**. Replacing a set of collinear features with their top principal components eliminates multicollinearity, stabilizing downstream linear models.

Handling Nonlinear Data

Standard PCA is a strictly **linear transformation** and cannot capture complex, non-linear relationships. To address non-linear structures, you must use **Kernel PCA**, which applies the kernel trick to map data into a higher-dimensional space where linear separations can be performed.

Choosing Feature Selection Over PCA

You should prefer discrete feature selection (like Lasso or recursive feature elimination) over PCA when:

- **Model Interpretability is Required:** PCA blends the original features into abstract linear combinations, making it difficult to explain what a specific component represents in the real world. Feature selection retains the original variables intact.
- **Computational Constraints exist at Inference:** PCA requires transforming every new data point through a projection matrix, whereas feature selection allows you to stop collecting or measuring the dropped features entirely.

40. What are the main components of a neural network? What is the role of weights and biases? What is a layer in a neural network? What are input, hidden, and output layers? What is

backpropagation? What is loss function in neural networks for regression and classification problems?

Main Components

A Neural Network (Multilayer Perceptron) consists of interconnected computational nodes called **neurons** arranged sequentially in structural structural groups called **layers**.

Role of Weights and Biases

- **Weights (w):** Multiplicative parameters that control the signal strength or relative importance of an incoming input feature. They are adjusted during training to learn patterns.
- **Biases (b):** Additive parameters that shift the activation function's output up or down, allowing the neuron to fit patterns that do not pass directly through the coordinate origin.

Layer Hierarchy

- **Layer:** A collection of parallel neurons that process inputs from a preceding stage simultaneously.
- **Input Layer:** The initial entry point that receives raw feature data from the dataset and passes it into the network.
- **Hidden Layers:** Intermediate layers between the input and output layers. They extract abstract, high-level features and non-linear representations from the input data.
- **Output Layer:** The final layer that produces the model's concrete predictions (e.g., continuous values for regression or probability scores for classification).

Backpropagation Definition

Backpropagation is the algorithm used to train neural networks. It calculates the gradient of the loss function with respect to every weight and bias in the network by applying the calculus **chain rule** from the output layer backward through the hidden layers. These calculated gradients are then used by an optimizer to update the network parameters and minimize error.

Loss Functions

- **For Regression:** Mean Squared Error (MSE) or Mean Absolute Error (MAE), which penalize distance errors.
- **For Classification: Binary Cross-Entropy** for two-class problems, or **Categorical Cross-Entropy** for multi-class problems, which penalize deviations from true class probabilities.

41. What is an activation function? What is epoch, batch size, and iteration in NN? What is chain rule in backpropagation?

Why do we compute derivatives in training?

Activation Function Purpose

An **activation function** is a mathematical formula applied to the weighted sum of a neuron's inputs:

$$\text{Output} = f\left(\sum w_i x_i + b\right)$$

Without activation functions, a neural network is just a series of linear combinations, which simplifies mathematically to a basic linear regression model regardless of how many layers you add. Activation functions introduce **non-linearity**, allowing the network to learn complex, non-linear relationships in the data.

Structural Training Parameters

- **Epoch:** One complete pass where the optimization loop views the entire training dataset exactly once.
- **Batch Size:** The number of training samples processed together in a single forward and backward pass before the model updates its internal weights.
- **Iteration:** The total number of batch updates completed. For example, if a dataset has $1,000$ samples and the batch size is 100, it takes 10 iterations to complete 1 full epoch.

The Chain Rule in Backpropagation

Because a neural network is a nested composition of functions (each layer feeding into the next), calculating how a weight in an early layer affects the final output error requires the calculus **chain rule**.

The chain rule breaks down the derivative of a composite function into a product of individual derivatives:

$$\frac{\partial \text{Loss}}{\partial w_{\text{hidden}}} = \frac{\partial \text{Loss}}{\partial \text{Output}} \times \frac{\partial \text{Output}}{\partial \text{Hidden}} \times \frac{\partial \text{Hidden}}{\partial w_{\text{hidden}}}$$

This allows the network to propagate error signals backward through its layers to update early weights accurately.

Why We Must Compute Derivatives

We compute derivatives because they provide the **gradient**—the vector showing the direction and rate of fastest increase of the loss function. By taking the derivative of the loss function with respect to a weight ($\frac{\partial \text{Loss}}{\partial w}$), the optimization algorithm learns exactly how to adjust that

parameter (increasing or decreasing it) to reduce overall error and guide the model toward convergence.